Report Homework Assignment 3

Mursal Stella Mohammadi

November 9, 2026

1 Introduction

This assignments goal is to find the greatest number in a large integer array by implementing and evaluating a sequential version and a parallel version written in CUDA C/C++ and executed on an NVIDIA GPU. For the parallel version the described reduction tree logic needs to be used.

GPU

A Graphics Processing Unit (GPU) originally, as its name suggests, was used to enhance rendering 3D graphics and images. Its computational power is also used to solve general computational problems, like searching, sorting instead of graphics. This is known as General Purpose computing on GPU (GPGPU). A GPU is used for parallelism using a SIMD (Single Instruction, Multiple Data or SIMT (Single Instruction, Multiple Thread) execution model. NVIDIA GPUs are built from multiple Streaming Multiprocessors (SMs), each containing multiple control units and a large collection of execution datapaths called Streaming Processors (SPs) or cores.

SIMT

GPUs follow a Single Instruction, Multiple Thread (SIMT) execution model. In SIMT, multiple threads run the same instruction simultaneously across different data. To hide memory access latency, some threads may block while memory is accessed and other threads, that have already accessed the data, may proceed with execution.

Threads, Blocks, Grids and Warps

CUDA organizes parallel execution into four connected levels

- Thread: The smallest execution unit, running one copy of the kernel code.
- Blocks: A group of threads (like 256 or 512 threads) assigned to run on a single SM. Threads in the same block can share local data and synchronize.
- Grid: The complete set of all thread blocks launched by a single kernel call.
- Warp: A group of 32 consecutive threads within a block that execute instructions together in lockstep.

Core (SP) and Streaming Multiprocessor (SM)

An NVIDIA GPU is divided into multiple hardware processing units called Streaming Multiprocessors (SMs). Each SM contains a collection of execution cores called Streaming Processors (SPs). A powerful Nvidia processor has 82 SMs, and each SM has 128 SPs for a total of 10,496 SPs. Each SM has a small block of memory that is shared among its SPs.

GPU Global Memory, Shared Memory, Registers

GPU memory is organized into three main layers with different speeds and access levels:

- Global Memory: The largest memory space on the GPU, accessible by all threads across all blocks, but with much higher access latency
- Shared Memory: Fast, low-latency memory shared exclusively by threads within the same block.
- Registers: Private per-thread storage and the fastest memory on the GPU.

[2]

2 Method

2.1 Idea: reduction tree

The serial version needs $n-1$ comparisons, and each comparison depends on the previous running maximum. This chain cannot run in parallel. A reduction tree removes the dependency: the values are compared pairwise, the winners are compared pairwise again, and so on until one value remains. All comparisons within one level are independent and run at the same time. n values need $\log_2 n$ levels instead of $n-1$ sequential steps.

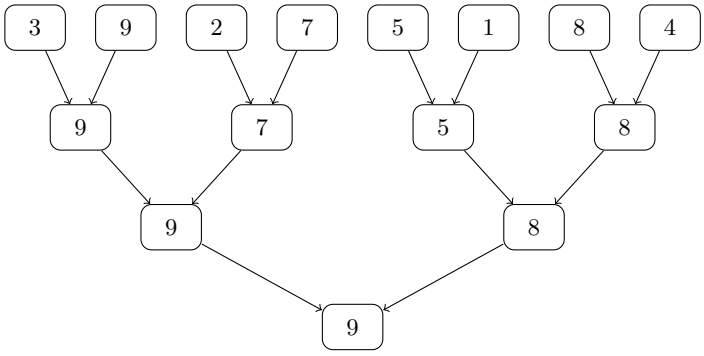


Figure 1: Reduction tree for eight values: three levels instead of seven sequential comparisons.

2.2 Distribution of the array

The array has more elements than the GPU has threads. Each thread therefore scans several elements spaced one grid width apart (grid-stride loop) and keeps its own maximum. This gives every thread almost the same amount of work, and the result is correct for any array length and any number of threads. Threads that receive no element keep the neutral value INT_MIN, which never wins a comparison. After this step every thread holds exactly one value, and these values are the leaves of the tree.

2.3 Three levels of the tree

The tree is mapped onto the three levels of the GPU thread hierarchy.

Level	Combines	Communication	Result
1. Warp	32 threads of a warp	registers (warp shuffle)	one value per warp
2. Block	8 warps of a block	shared memory and barrier	one value per block
3. Grid	1024 blocks	atomic operation in global memory	one final value

Table 1: Mapping of the reduction tree to the GPU hierarchy.

Level 1: warp. The 32 threads of a warp exchange values directly between registers. In five steps (distance 16, 8, 4, 2, 1) the number of candidate values halves each time, and the first thread of the warp ends with the warp maximum. [2]

Level 2: block. A block of 256 threads contains 8 warps, so 8 warp maxima exist. Warps cannot exchange registers, so each warp writes its maximum to shared memory. After all warps have written, the first warp reads the 8 values and applies the warp reduction once more. The first thread of the block ends with the block maximum.

Level 3: global. Blocks cannot communicate during a kernel. Each block therefore combines its maximum into one variable in global memory with an atomic maximum operation, which performs read, compare and write as one indivisible step. Only 1024 atomic operations occur in total, one per block. The variable is initialised with `INT_MIN` before every run.

2.4 Launch configuration

All experiments use 1024 blocks of 256 threads (8 warps per block). The grid-stride loop keeps the result correct for any configuration, so the configuration only affects speed. The same fixed configuration on both nodes keeps the hardware as the only difference between the two environments. [1]

3 Implementation

3.1 Overview

The serial version is a single loop over the array with a running maximum. The parallel version is one CUDA kernel, `kernelFindMax`, with a device helper `warpFindMax`. The pseudocode shows the structure of the kernel:

```
each thread:
    m = -INF
    for i = global_id; i < n; i += total_threads:
        m = max(m, a[i])                      // grid-stride loop

warp level:
    for offset in 16, 8, 4, 2, 1:
        m = max(m, shuffle_down(m, offset))  // lane 0 holds warp max

block level:
    if lane == 0: shared[warp_id] = m
    barrier
    warp 0: m = shared[lane] (or -INF), repeat warp reduction
```

```
global level:
    thread 0 of each block: atomicMax(result, m)
```

3.2 Warp level

Inside a warp, values are exchanged directly between registers:

```
int neighbour = __shfl_down_sync(0xffffffff, val, offset);
val = (val > neighbour) ? val : neighbour;
```

The mask `0xffffffff` declares that all 32 lanes take part. All 32 threads of a warp must reach this call. Threads without data therefore do not return early. They carry `INT_MIN` and execute the call as well.

3.3 Block level

Shuffle instructions do not cross warp boundaries. Each warp therefore writes its maximum to a shared array, and the first warp reduces these values:

```
if (lane == 0) warpMax[warpId] = localMax;
__syncthreads();
```

The barrier guarantees that all warps have written their value before warp 0 reads. It must be reached by all threads of the block, so it is placed outside of any condition.

3.4 Global level

Blocks cannot synchronize within one kernel launch. Thread 0 of each block combines its result with the global result using one atomic operation:

```
atomicMax(max, val);
```

The read-compare-write is performed as one indivisible step, so concurrent updates from different blocks cannot overwrite each other. With 1024 blocks, 1024 atomic operations occur in total. The result variable is set to `INT_MIN` on the host before every launch.

3.5 Host code and timing

For every run the host

1. times the serial version with `clock_gettime(CLOCK_MONOTONIC)`,
2. copies the array and the initial value `INT_MIN` to the device,
3. launches `kernelFindMax<<<1024, 256>>>`,
4. copies the result back and compares it with the serial result.

CUDA events surround the host-to-device copy, the kernel and the device-to-host copy separately. The total parallel time is the sum of the three. Allocation and freeing of memory are not measured. Each experiment is repeated 30 times.

3.6 Build and run

The program is compiled with `make main`, which calls `nvcc` for compute capability 7.5 and 8.6. It is started with `./main SMALL`, `./main MIDDLE` or `./main LARGE` for 2^{27}, 2^{28} and $2^{27} + 2^{28}$ elements. The output is written to `results_<hostname>_<SIZE>.txt`. Details are in the README.

4 Experimentations and results

Hardware and System Environment

All measurements were carried out on the IFI cluster. I used the nodes c6-0 and c6-4 found through the assignements hint /share/ifi/list-cuda-hosts.sh through python3 scr.py also given through assignemnt i got the compute capability. All three array sizes (0.5, 1.0 and 1.5 GiB) were measured on both nodes.

Table 2: Hardware and environment comparison of the two cluster nodes.

Category	**c6-0**	**c6-4**
CPU		
Model	Intel(R) Xeon(R) CPU E5-1620 v2	Intel(R) Core(TM) i9-10850K
Clock (base)	3.70 GHz	3.60 GHz
Clock (max / boost)	3.70 GHz	5.20 GHz
System Memory	62 GiB	125 GiB
GPU		
Model	TU106 [GeForce RTX 2060 Rev. A]	GA102 [GeForce RTX 3090]
Clock (base SM)	300 MHz	210 MHz
Clock (max SM / boost)	2130 MHz	2130 MHz
Clock (max memory)	7001 MHz	9751 MHz
VRAM	6144 MiB	24576 MiB
Cores	1920	10496
SMs	30	82
Operating System		
OS	AlmaLinux 9.8 (Olive Jaguar)	AlmaLinux 9.8 (Olive Jaguar)

Results for all array sizes

The tables and the figure show the results for all three array sizes. Serial and parallel time grow linearly with the array size on both nodes. Doubling the size from 0.5 to 1.0 GiB doubles the serial time on both nodes (factor 2.0) and the parallel time (c6-4: 1.99, c6-0: 2.02). The 1.5 GiB array needs about three times the time of the 0.5 GiB array (serial: 2.89 on c6-4 and 2.99 on c6-0; parallel: 2.99 and 3.01).

The host-to-device copy takes 98.5–98.6 % (c6-4) and 97.1–97.2 % (c6-0) of the parallel time at every size. The kernel time also grows approximately linearly (c6-4: 0.74, 1.44, 2.16 ms; c6-0: 2.11, 4.32, 6.66 ms).

Because the copy grows at the same rate as the serial scan, the speedup does not increase with the array size. It stays near 3.2 on c6-4 (3.16, 3.17, 3.05) and near 3.3 on c6-0 (3.28, 3.24, 3.26). On c6-4 the speedup for 1.5 GiB is lower because the serial time (465.6 ms) is below the linear extrapolation of the smaller sizes (about 484 ms), while the parallel time scales linearly.

The runs are very stable. Minimum and mean are close at all sizes. Efficiency is speedup divided by the number of SMs, which explains the higher values on c6-0 (30 SMs) than on c6-4 (82 SMs).

Table 3: Mean elapsed times over 30 runs for all array sizes. Times in ms. Efficiency is speedup divided by the number of SMs (82 on c6-4, 30 on c6-0).

	c6-4 (RTX 3090)			**c6-0 (RTX 2060)**		
Array size (GiB)	0.5	1.0	1.5	0.5	1.0	1.5
Serial	161.361	322.131	465.580	250.846	500.665	750.859
Parallel, total	51.112	101.473	152.820	76.452	154.628	230.275
Host to device	50.363	100.022	150.652	74.330	150.296	223.597
Kernel	0.739	1.440	2.157	2.108	4.319	6.664
Device to host	0.011	0.011	0.011	0.014	0.013	0.014
Parallel / serial (%)	31.68	31.50	32.82	30.48	30.88	30.67
Speedup	3.157	3.175	3.047	3.281	3.238	3.261
Efficiency (%)	3.85	3.87	3.72	10.94	10.79	10.87

Table 4: Minimum elapsed times over 30 runs for all array sizes. Times in ms.

	c6-4 (RTX 3090)			**c6-0 (RTX 2060)**		
Array size (GiB)	0.5	1.0	1.5	0.5	1.0	1.5
Serial	160.915	321.460	464.499	250.681	500.061	750.676
Parallel, total	50.811	101.353	152.468	76.318	154.432	230.138
Host to device	50.068	99.910	150.305	74.199	150.190	223.494
Kernel	0.731	1.434	2.148	2.087	4.220	6.602
Device to host	0.007	0.006	0.006	0.010	0.009	0.010
Parallel / serial (%)	31.58	31.53	32.82	30.44	30.88	30.66
Speedup	3.167	3.172	3.047	3.285	3.238	3.262
Efficiency (%)	3.86	3.87	3.72	10.95	10.79	10.87

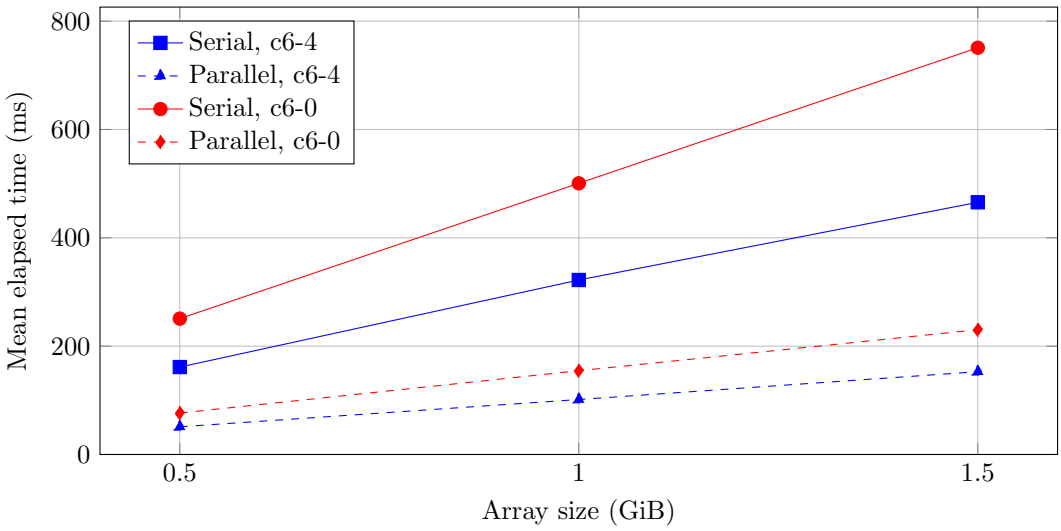


Figure 2: Mean elapsed time over 30 runs for serial and parallel version on both nodes.

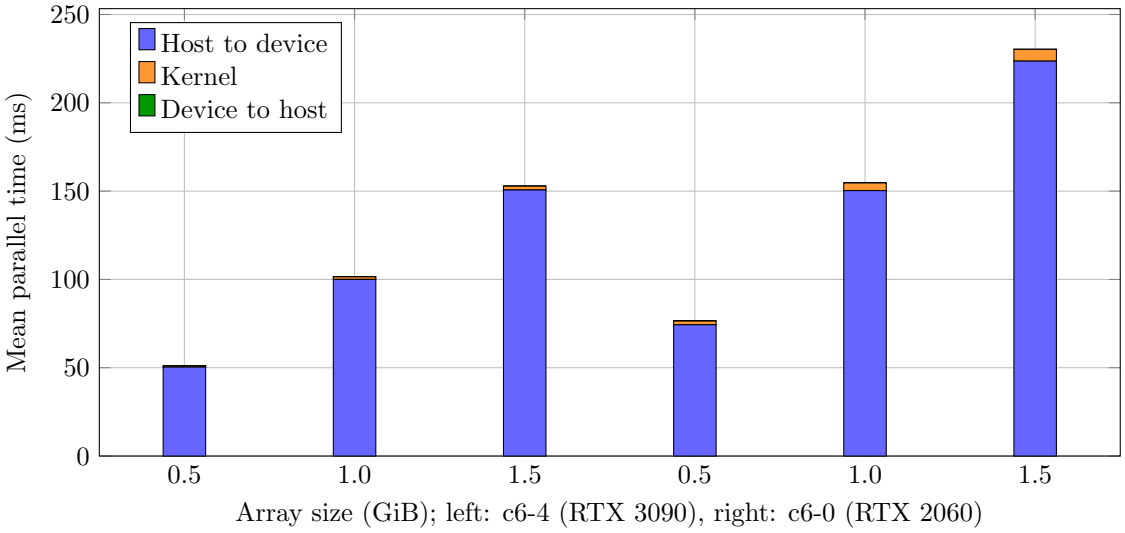


Figure 3: Mean parallel time split into host-to-device copy, kernel and device-to-host copy. The kernel and the device-to-host segments are barely visible because the host-to-device copy takes 97–99 % of the time.

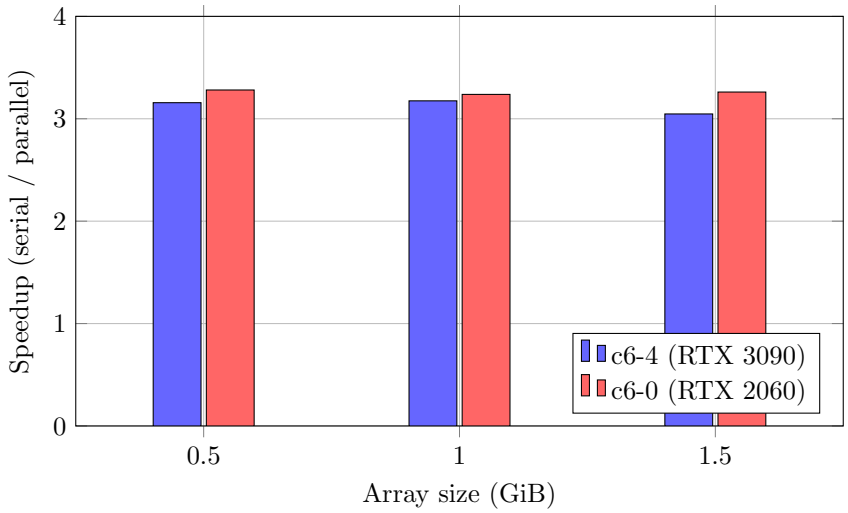


Figure 4: Speedup of the parallel version over the serial version (mean of 30 runs).

5 Discussion

On both nodes the GPU result matched the serial maximum. The major time-consuming part is the host-to-device copy. It takes 98.6 % (c6-4) and 97.1 % (c6-0) of the mean parallel time, while the kernel takes only 1.4 % and 2.9 %. The kernel alone is faster than the serial loop, but the copy limits the total speedup. The RTX 3090 node (c6-4) is faster in every parallel component (kernel 2.157 ms against 6.664 ms). The speedup is nevertheless higher on c6-0 (3.26 against 3.05), because its CPU makes the serial version slower (750.9 ms against 465.6 ms). A possible improvement could maybe using streams.

6 Discussion

On both nodes the GPU result matched the serial maximum in every run. The host-to-device copy is the major time-consuming part. It takes 98.5–98.6 % (c6-4) and 97.1–97.2 % (c6-0) of the parallel time at every array size, while the kernel takes only 1.4 % and 2.9 %. The kernel alone is about 216 times (c6-4) and 113 times (c6-0) faster than the serial loop at 1.5 GiB, but the copy limits the total speedup to about 3. Only one comparison is done per loaded element, so the problem is transfer-bound, not compute-bound. Since the copy grows with the array size at the same rate as the serial scan, the speedup does not improve for larger arrays (Figure 4).

7 Conclusion

In this assignment, a serial and a parallel CUDA version for finding the greatest number in an integer array were implemented and compared. The parallel version uses a three-level reduction tree:. warp shuffles inside each warp, shared memory and `__syncthreads()` inside each block, and `atomicMax` across blocks. A grid-stride loop distributes the work evenly. On both nodes the result matched the serial maximum.

For all three array sizes, the parallel version needed 31–33 % (RTX 3090, c6-4) and 30–31 % (RTX 2060, c6-0) of the serial time, which corresponds to speedups of 3.05–3.18 and 3.24–3.28. Serial and parallel time scale linearly with the array size, so the speedup stays constant. The host-to-device copy takes 97–99 % of the parallel time and limits the speedup.

References

[1] NVIDIA Corporation: *CUDA C++ Programming Guide*. NVIDIA Corporation, 2026. URL: `https://docs.nvidia.com/cuda/cuda-programming-guide/contents.html`.

[2] Peter S. Pacheco and Matthew Malensek: *An Introduction to Parallel Programming*. Second Edition. Morgan Kaufmann (imprint of Elsevier), Cambridge, MA, United States, 2022. ISBN: 978-0-12-804605-0.